Code Layout, Readability and Reusability

Phase 5 | Maintainability review

Repository layout

Path	Responsibility
frontend/src/pages/	Page-level account, planning, and trip experiences.
frontend/src/components/	Reusable trip form and itinerary presentation.
frontend/src/services/api.js	HTTP and local session/cache operations.
backend/app/routers/ and routes/	API registration and endpoint implementation.
backend/app/schemas/	Pydantic request and response contracts.
backend/app/services/	Authentication, AI, trip persistence, Supabase, and history logic.
backend/app/prompts/ and services/destination_data.py	Prompt templates and local destination/day generation.
backend/tests/	Offline reliability and configuration scenarios.

Positive structure

- UI pages are separated from the API service module.
- FastAPI routes use schemas and dependency-injected services.
- Provider behavior is abstracted behind an AI orchestrator/provider interface.
- Environment configuration and example files support separate runtime settings.
- The trip router module re-exports the route implementation for modular compatibility.

Improvement opportunities

- Reduce duplicated or legacy modules after confirming import compatibility.
- Resolve the duplicate SQL schema copies and document a single source of truth.
- Add frontend automated tests and end-to-end tests.
- Extract shared API error handling and validation where repeated.
- Add explicit types/contracts for frontend API payloads to reduce drift.
- Review log files and generated artifacts so they do not become accidental source deliverables.

Assessment boundary

This is a source-layout review, not a formal code-quality score, static-analysis report, or measured maintainability evaluation.

Prepared from the current project source and configuration. Product assumptions and proposed work are labeled; no user research, test result, or deployment result is implied unless explicitly stated.